Supervyse OPF Redfish SSE & Redfish Event Listener User Guide

Revision 1.0
12 2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1 Introduction 5
1.1 Overview 5
1.2 Related Information 5
2 Server Sent Events 6
2.1 Introduction 6
2.1.1 SSE Stream Format 7
2.1.1.1 Metric Report SSE Stream 7
2.1.1.2 Event message SSE stream 8
2.1.2 SSE Filter Query Parameter Usage 8
3 Receive Events Or Metric Reports by SSE 10
3.1 Events 10
3.2 Metric Repots 10
4 Redfish Event Listener 12
4.1 Introduction 12
4.2 Configure Introduction 12
4.3 Sample Configuration 14
5 Running Redfish Event Listener 15

Revision History

Revision Number	Description	Author	Reviewer	Release Date
1.0	Initial release.	Ezra Lin	Victor Wu	12, 2024

1 Introduction

The document aims to help users understand how to use the SSE and Redfish Event Listener.

1.1 Overview

The document aims to help users understand how to use the SSE and Redfish Event Listener. The SSE section introduces how to use the SSE to receive events or metric reports. The Redfish Event Listener section introduces how to use this DMTF Tool to receive events or metric reports.

1.2 Related Information

The following publications and sources of information are referred to by this document or may be useful to you:

- *Redfish Event Listener, DMTF, 2024, https://github.com/DMTF/Redfish-Event-Listener*
- *Redfish Specification, DMTF, 2024, https://www.dmtf.org/sites/default/files/standards/documents/DSP0266_1.21.0.pdf*
- *Supervyse OPF Redfish Api User Guide, 1.38, Insyde Software, 2024*

2 Server Sent Events

Server-sent events (SSE), defined by the Web Hypertext Application Technology Working Group (WHATWG), enables a client to open a connection with a web service. The web service can continuously push data to the client, as needed.

2.1 Introduction

The ServerSentEventUri and SSEFilterPropertiesSupported properties in the EventService resource can be used to determine the SSE URL and which "$filter" query parameters are supported by SSE.

Here is the EventService resource example.

The SSE's URL is "/redfish/v1/EventService/SSE". SSE supports the RegistryPrefix EventFormatType, MessageId, and MetricReportDefinition filter query parameters.


```
{
   "@odata.id": "/redfish/v1/EventService",
   "@odata.type": "#EventService.v1_10_0.EventService",
   "Actions": {
      "#EventService.SubmitTestEvent": {
         "target": "/redfish/v1/EventService/Actions/EventService.SubmitTestEvent"
      },
      "#EventService.TestEventSubscription": {
         "target": "/redfish/v1/EventService/Actions/EventService.TestEventSubscription"
      }
   },
   "DeliveryRetryAttempts": 3,
   "DeliveryRetryIntervalSeconds": 30,
   "EventFormatTypes": [
      "Event",
      "MetricReport"
   ],
   "ExcludeMessageId": [],
   "Id": "EventService",
   "Name": "Event Service",
   "RegistryPrefixes": [
      "OpenBMC",
      "TaskEvent"
   ],
   "ResourceTypes": [
      "Task"
   ],
   "SMTP": {
      "Authentication": "None",
      "ConnectionProtocol": "None",
      "FromAddress": "",
      "Password": null,
      "Port": 0,
```

```
      "ServerAddress": "",
      "Username": ""
   },
   "SSEFilterPropertiesSupported": {
      "EventFormatType": true,
      "ExcludeRegistryPrefix": false,
      "MessageId": true,
      "MetricReportDefinition": true,
      "OriginResource": false,
      "RegistryPrefix": true,
      "ResourceType": false
   },
   "ServerSentEventUri": "/redfish/v1/EventService/SSE",
   "ServiceEnabled": true,
   "Status": {
      "State": "Enabled"
   },
   "Subscriptions": {
      "@odata.id": "/redfish/v1/EventService/Subscriptions"
   }
}
```


2.1.1 SSE Stream Format

The service shall use the data field in the SSE stream based on the payload format. The SSE streams have these formats:

- Metric report SSE stream
- Event message SSE stream

2.1.1.1 Metric Report SSE Stream

The service shall use the data field in the SSE stream to include the JSON representation of the MetricReport.

Here is the Metric report SSE stream which sends to the client's SSE connection.


```
id: 74
data: {
data:   "@odata.id": "/redfish/v1/TelemetryService/MetricReports/MyMetric",
data:   "@odata.type": "#MetricReport.v1_5_0.MetricReport",
data:   "Id": "MyMetric",
data:   "MetricReportDefinition": {
data:     "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/MyMetric"
data:   },
data:   "MetricValues": [
data:     {
data:       "MetricProperty":
"/redfish/v1/Chassis/BNC_Baseboard/Sensors/voltage_System_Fans_SE_12V_Output_Voltage",
data:       "MetricValue": "12.056000",
data:       "Timestamp": "2024-12-18T07:56:59.493+00:00"
```

```
data:     }
data:   ],
data:   "Name": "MyMetric",
data:   "Timestamp": "2024-12-18T07:56:59.493+00:00"
data: }
```

2.1.1.2 Event message SSE stream

The service shall use the data field in the SSE stream to include the JSON representation of the Event object.

Here is the Event message SSE stream which sends to the client's SSE connection.

```
id: 1
data: {
data:   "@odata.type": "#Event.v1_8_0.Event",
data:   "Events": [
data:     {
data:       "Context": "",
data:       "EventId": "1734506047",
data:       "EventTimestamp": "2024-12-18T07:14:07+00:00",
data:       "EventType": "Event",
data:       "Message": "Event subscription with id 915325207 was added.",
data:       "MessageArgs": [
data:         "915325207"
data:       ],
data:       "MessageId": "OpenBMC.0.1.EventSubscriptionAdded",
data:       "Severity": "OK"
data:     }
data:   ],
data:   "Id": "1",
data:   "Name": "Event Log"
data: }
```

2.1.2 SSE Filter Query Parameter Usage

Below is the usage and introduction of SSE filter query parameters.

Note: The SSEFilterPropertiesSupported property indicates that the service supports SSE filter query parameters in the EventService resource, please refer to it first.

- EventFormatType

The service sends events of the matching EventFormatType.

Example:

```
https://sseuri?$filter=EventFormatType eq 'Event'
```

Valid values are the EventFormatType enumerated string values that the Redfish EventService schema defines.

- MessageId

The service sends events with the matching MessageId.

Example:

```
https://sseuri?$filter=MessageId eq 'Contoso.1.0.TempAssert'
```

- MetricReportDefinition

The service sends metric reports generated from the MetricReportDefinition.

Example:

```
https://sseuri?$filter=MetricReportDefinition eq
'/redfish/v1/TelemetryService/MetricReportDefinitions/PowerMetrics'
```

- OriginResource

The service sends events for the resource.

Example:

```
https://sseuri?$filter=OriginResource eq '/redfish/v1/Chassis/1/Thermal'
```

- RegistryPrefix

The service sends events with messages that are part of the RegistryPrefix.

Example:

```
https://sseuri?$filter=(RegistryPrefix eq 'Resource') or (RegistryPrefix eq 'Task')
```

- ResourceType

The service sends events for resources that match the ResourceType.

Example:

```
https://sseuri?$filter=(ResourceType eq 'Power') or (ResourceType eq 'Thermal')
```

- SubordinateResources

When SubordinateResources is true and OriginResource is specified, the service sends events for the resource and its subordinate resources.

Example:

```
https://sseuri?$filter=(OriginResource eq '/redfish/v1/Systems/1') and (SubordinateResources eq
true)
```

3 Receive Events Or Metric Reports by SSE

This section will introduce using the Browser(Chrome) to receive events or metric reports via SSE.

3.1 Events

Here is the received events example:

Input "https://<Service IP>/redfish/v1/EventService/SSE?$filter=EventFormatType eq Event" at Browser search bar.

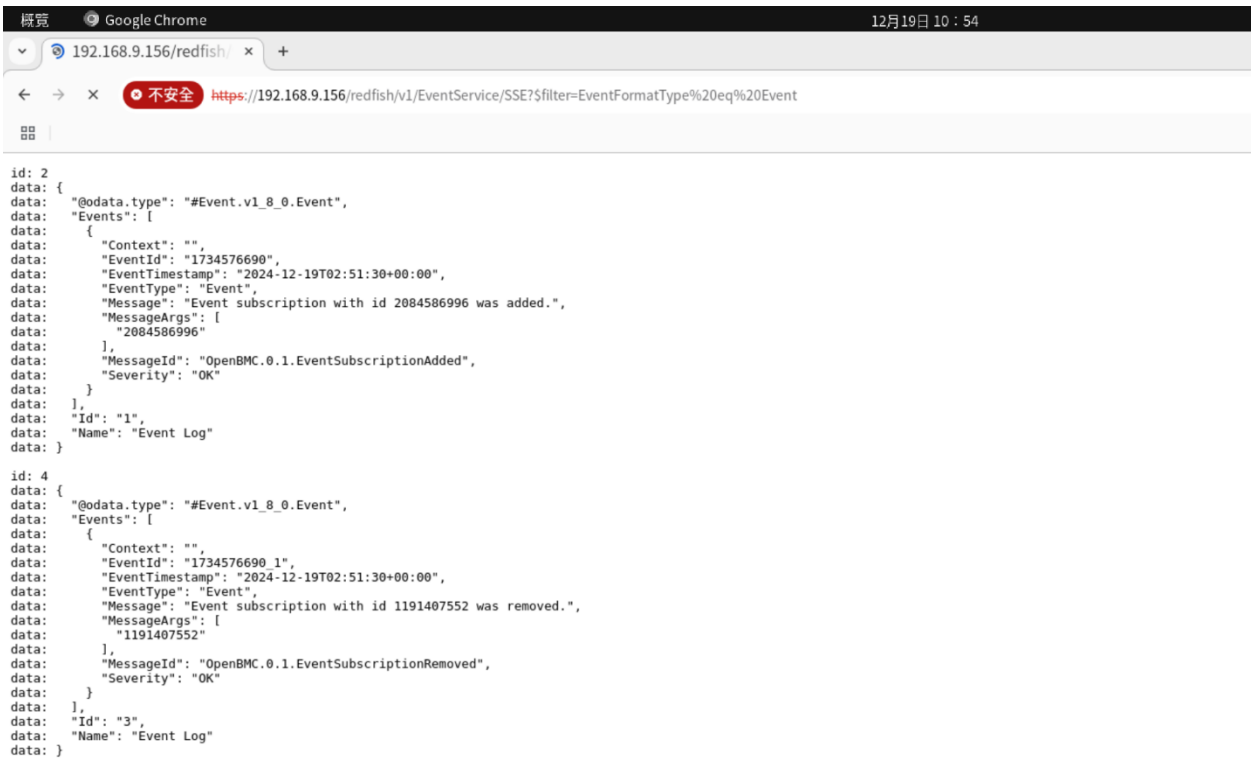

Figure 3-1 Event

3.2 Metric Repots

You need to create the MetricReportDefination resource first if you want to receive metric reports.

Please refer to Supervyse OPF Redfish Api User Guide to create a MetricReportDefination resource.

Here is the received metric reports example:

Input "https://<Service IP>/redfish/v1/EventService/SSE?$filter=EventFormatType eq MetricReport" at Browser search bar.

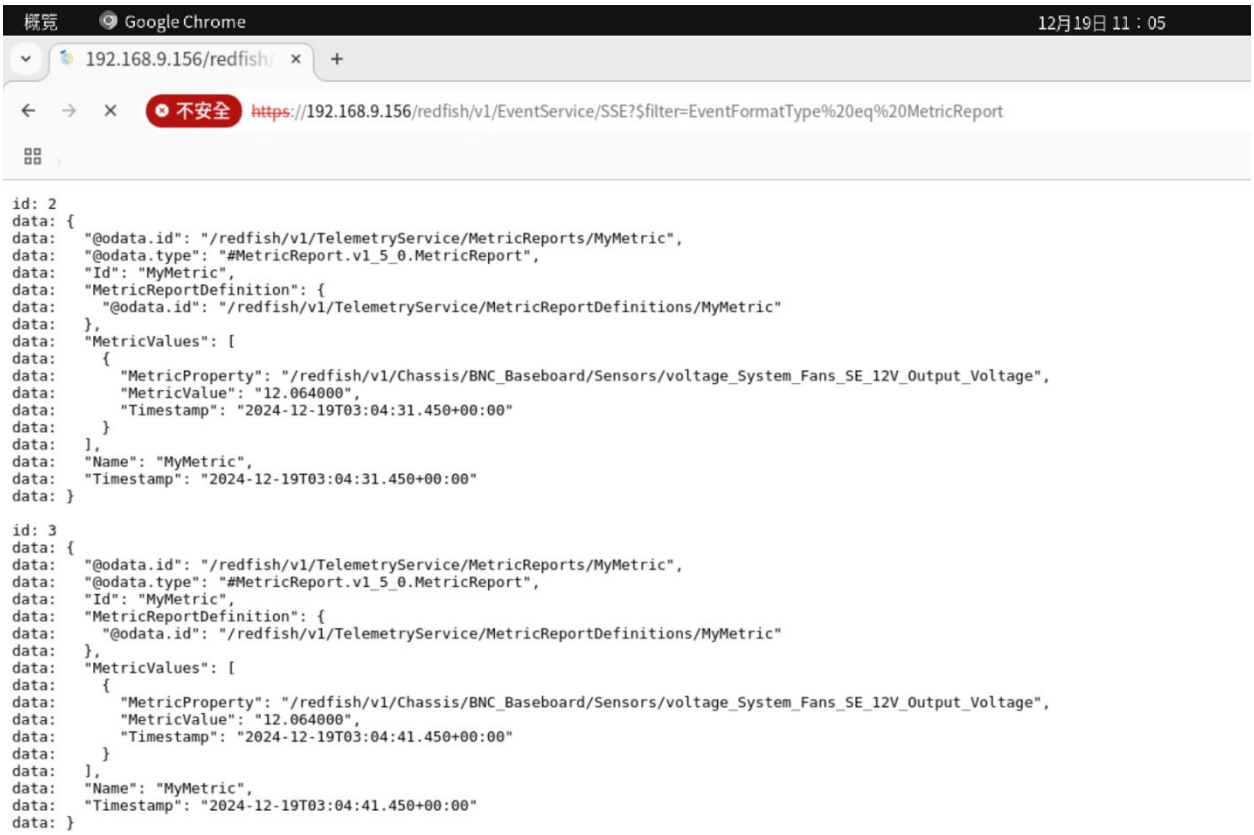

Figure 3-2 Metric

4 Redfish Event Listener

4.1 Introduction

The Redfish Event Listener is a lightweight HTTP(S) server that can be deployed to read and record events from Redfish services. It can also subscribe to multiple services.

4.2 Configure Introduction

This section will introduce the Redfish Event Listener configuration.

The file config.ini contains a default configuration for the tool. This can be used as a template for real configurations or edited as needed.

The Information Table contains details about the configuration file itself. The tool does not parse any information in this section, and is used to help users track configuration files.

Table 4.2-1 Information Table

	Description
Upload	The date the file was updated.
Description	A description of the configuration.

The SystemInformation Table contains settings for the system acting as the event listener. All options in this section are required.

Table 4.2-2 SystemInformation Table

	Description
ListenerIP	The interface to use to listen for events. This is typically 0.0.0.0 to listen on all interfaces for IPv4 or :: to listen on all interfaces for IPv6.
ListenerPort	The port number to use to listen for events. This is typically 80 for HTTP or 443 for HTTPS.
UseSSL	on to use HTTPS, off to use HTTP for incoming event messages. If HTTPS is to be used, the CertificateDetails section will need to be configured.

The CertificateDetails Table contains the certificate information for the system acting as the event listener. This section and its options are only required if UseSSL in SystemInformation is set to on.

Table 4.2-3 CertificateDetails Table

	Description
certfile	The file containing the certificate for the event listener. The tool comes with a default self-signed certificate named cert.pem.
keyfile	The file containing the certificate private key for the event listener. The tool comes with a default private key named server.key.

The SubscriptionDetails Table contains subscription information so that the Redfish service can send the desired events to the event listener. The Destination option is required; other options can be omitted or have an empty value.

All options can be mapped to the property in the EventDestination resource. Please refer to Supervyse OPF Redfish Api User Guide for more information about the EventDestination resource.

Table 4.2-4 SubscriptionDetails Table

	Description	Mapping Property in EventDestination Resource
Destination	The URI of the event listener. The Redfish service will perform an HTTP POST on this URI to send events to the event listener.	Destination
Context	The format of the event notifications the event listener will receive. Possible values are Event and MetricReport	Context
Format	The format of the event notifications the event listener will receive. Possible values are Event and MetricReport. The EventFormatTypes property in the EventService resource indicates the formats currently supported.	EventFormatType
Expand	true indicates if the OriginOfCondition property will contain the expanded resource in the event payload. false indicates no payload expansion.	IncludeOriginOfCondition
ResourceTypes	An array of resource types the Redfish service will use to filter events for the event listener. For OPF G4 example include "Task". The ResourceTypes property in the EventService resource indicates the formats currently supported.	ResourceTypes
Registries	An array of registry names the Redfish service will use to filter events for the event listener.	RegistryPrefixes

	The RegistryPrefixes property in the EventService resource indicates the formats currently supported.	

The ServerInformation Table contains information about the Redfish services that will transmit events to the event listener. All options in this section, except LoginType, are required.

Table 4.2-5 ServerInformation Table

	Description
ServerIPs	An array of URIs of the Redfish services for subscribing.
UserNames	An array of Redfish usernames for each of the listed Redfish services. This must be the same array length as ServerIPs.
Passwords	An array of Redfish passwords for each of the listed Redfish services. This must be the same array length as ServerIPs.
LoginType	An array of login types for each of the listed Redfish services. Possible values are Basic and Session. If not specified, Session is used.

4.3 Sample Configuration

The following example shows a configuration file with some of the options supported by the tool; This example is used to receive events.

```
[Information]
Updated = February 24, 2023
Description = Redfish Event Listener Tool Simple Config

[SystemInformation]
ListenerIP = 192.168.11.138
ListenerPort = 1968
UseSSL = on

[CertificateDetails]
certfile = cert.pem
keyfile = server.key

[SubscriptionDetails]
Destination = https://192.168.11.138:1968/test
Context = Public
Format = Event


[ServerInformation]
ServerIPs = ["https://192.168.9.156"]
UserNames = ["<username>"]
Passwords = ["<password>"]
LoginType = ["Basic"]
```

5 Running Redfish Event Listener

After configuring the setting, execute the below command to run Redfish Event Listener.

```
python3 RedfishEventListener_v1.py -v
```

You will see events on the terminal.

```
ezra.lin@ss1-HP348-G4-Genoa: ~    ezra-lin@Ezra-Lin: ~/Redfish-Event-List···    ezra-lin@Ezra-Lin: ~/DevToolPrivate/Je···    ezra-lin@Ezra-Lin: /tftpboot    ezralin@insydesc2-WF2: ~
Response Time for GET to /redfish/v1/EventService: 0.21909929101821035 seconds.
Attempt 1 of /redfish/v1/EventService/Subscriptions
Response Time for POST to /redfish/v1/EventService/Subscriptions: 0.2164645619923249 seconds.
Subscription is successful for https://192.168.9.156, /redfish/v1/EventService/Subscriptions/605281854, /redfish/v1/Eve
ntService/Subscriptions/605281854
Continuing with Listener.
Listening for events...
Press Ctrl-C to close program

Event received from 192.168.9.156
  EventType: Event
  MessageId: OpenBMC.0.1.EventSubscriptionAdded
  EventId: 1734600469
  EventTimestamp: 2024-12-19T09:27:49+00:00
  Severity: OK
  Message: Event subscription with id 605281854 was added.
  MessageArgs: ['605281854']

Event Counter for Host 192.168.9.156 = 1

192.168.9.156 - - [19/Dec/2024 17:27:49] "POST /test HTTP/1.1" 204 -
^CShutting down the Redfish Event Listener
```

Figure 5-1 Redfish Event Listener Example